GF01 Tracker Dashboard — UI Team Integration Guide

> **Audience:** Web / UI Development Team
> **Goal:** Build a frontend dashboard that visualizes GPS tracker data from the GF01 receiver.
> **Date:** 2026-07-13

---

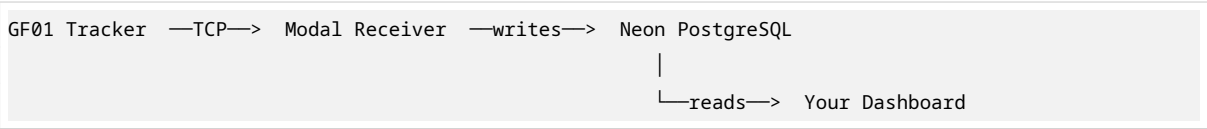
1. How the Backend Works

The GF01 receiver is a **long-running TCP server** (deployed on Modal) that listens for raw binary packets from GPS trackers. It does three things:

1. **Decodes packets** — GPS coordinates, WiFi access points, cell towers, battery, signal strength, SOS alarms, etc.
2. **Enriches location data** — When GPS is unavailable, it sends WiFi APs and cell towers to the **Google Maps Geolocation API** to get approximate lat/lon.
3. **Stores everything in PostgreSQL** — All packets and device metadata are persisted to a **Neon PostgreSQL** database.

Your dashboard frontend does **not** talk to the trackers directly. It only reads from the PostgreSQL database (or a lightweight backend API that wraps it).

Architecture



---

2. Database Schema

Two tables exist. Your dashboard will primarily read from these.

2.1 `tracker_devices`

One row per unique IMEI that has ever connected.

Column	Type	Description
`imei`	`TEXT` (PK)	Device IMEI (15 digits)
`first_seen`	`TIMESTAMPTZ`	First connection timestamp
`last_seen`	`TIMESTAMPTZ`	Most recent connection timestamp

Use this for the **device list**. A device is "online" if `last_seen` is within the last ~5 minutes.

2.2 `tracker_packets`

One row per packet received. This is your main data source.

Column	Type	Nullable	Description
id	BIGSERIAL	No	Auto-increment ID
imei	TEXT	Yes	Device IMEI
received_at	TIMESTAMPTZ	No	Server timestamp when packet arrived
packet_type	TEXT	Yes	Decoded type (see §3)
raw_hex	TEXT	No	Full raw packet as hex string
mcc	INTEGER	Yes	Mobile Country Code
mnc	INTEGER	Yes	Mobile Network Code
lac	INTEGER	Yes	Location Area Code
cell_id	BIGINT	Yes	Cell tower ID
latitude	DOUBLE PRECISION	Yes	Latitude (GPS or Google geolocation)
longitude	DOUBLE PRECISION	Yes	Longitude (GPS or Google geolocation)
speed	INTEGER	Yes	Speed in km/h
battery	INTEGER	Yes	Battery percentage
signal_strength	INTEGER	Yes	GSM signal strength
decoded_at	TIMESTAMPTZ	Yes	When processing finished

---

3. Packet Types You Care About

Not every packet has location data. Here is what each `packet_type` means for the UI.

packet_type	Has Location?	Accuracy	UI Purpose
gps	**Yes**	High (direct GPS fix)	**Primary map marker source**
wifi_lbs	**Yes**	Medium (Google API)	Fallback map markers
wifi_lbs_phone	**Yes**	Medium (Google API)	Same as above, plus phone number
status	No	—	Battery %, signal strength, firmware version
heartbeat	No	—	Keep-alive; no data
sos	No	—	**Emergency alarm** — show alert banner
drop_alarm	No	—	Fall detection alert
login	No	—	Device connected
manual_positioning	No	—	Device reported why GPS failed
battery_charged	No	—	Battery fully charged
charger_connected	No	—	Charger plugged in
charger_disconnected	No	—	Charger unplugged

Table 3 – continued

packet_type	Has Location?	Accuracy	UI Purpose
string	No	—	Arbitrary text from device
time_sync	No	—	Device requested time
sync_setup	No	—	Device requested configuration
unknown_0xXX	No	—	Unhandled protocol
error_0xXX	No	—	Server error during decoding

Key Rule for the Map

Only show markers where `latitude IS NOT NULL AND longitude IS NOT NULL`. Filter by `packet_type IN ('gps', 'wifi_lbs', 'wifi_lbs_phone')`.

- **Green markers** — `packet_type = 'gps'` (most accurate, from device GPS chip).
- **Blue markers** — `packet_type IN ('wifi_lbs', 'wifi_lbs_phone')` (Google-derived, less accurate).
- **Red markers / banner** — `packet_type = 'sos'` or `drop_alarm` (emergency, no coordinates but critical).

---

4. Recommended Dashboard Views

4.1 Map View (Primary)

What to show:

- All devices with valid `latitude`/`longitude` as markers.
- Different marker colors by `packet_type` and freshness (e.g., green if < 5 min old, yellow if < 30 min, red if older).
- Click a marker to open a card showing: IMEI, last seen, speed, battery, signal strength, fix type.
- For the selected device, draw a **polyline** connecting its historical points (last 24 hours).

Data query:

- Latest location per device: `SELECT imei, latitude, longitude, speed, packet_type, received_at FROM tracker_packets WHERE latitude IS NOT NULL ORDER BY received_at DESC LIMIT 1` per device (use `DISTINCT ON (imei)` or a lateral join).
- History: `SELECT received_at, latitude, longitude, speed, packet_type FROM tracker_packets WHERE imei = ? AND latitude IS NOT NULL AND received_at >= NOW() - INTERVAL '24 hours' ORDER BY received_at ASC`.

4.2 Device List Sidebar

What to show:

- IMEI, online/offline badge, last seen time (human-readable, e.g., “2m ago”).

- Clicking a device centers the map on it and loads its history.

Data query:

- `SELECT imei, first_seen, last_seen FROM tracker_devices ORDER BY last_seen DESC`.
- Online status: `NOW() - last_seen < INTERVAL '5 minutes'`.

4.3 Device Detail Panel

What to show:

- Current location (lat/lon), fix type, timestamp.
- Battery % and signal strength (from latest `status` packet).
- Speed and heading (from latest `gps` packet).
- Raw packet log (last 20 `raw_hex` values) for debugging.

Data queries:

- Latest status: `SELECT battery, signal_strength, received_at FROM tracker_packets WHERE imei = ? AND packet_type = 'status' ORDER BY received_at DESC LIMIT 1`.
- Latest GPS: `SELECT latitude, longitude, speed, received_at FROM tracker_packets WHERE imei = ? AND packet_type = 'gps' ORDER BY received_at DESC LIMIT 1`.
- Recent packets: `SELECT packet_type, received_at, raw_hex FROM tracker_packets WHERE imei = ? ORDER BY received_at DESC LIMIT 20`.

4.4 Alerts Panel

What to show:

- A persistent banner or badge for `sos` and `drop_alarm` events.
- Sound notification or browser push notification on new SOS.
- Filterable history of all alarm events.

Data query:

- `SELECT imei, packet_type, received_at, raw_hex FROM tracker_packets WHERE packet_type IN ('sos', 'drop_alarm') AND received_at >= NOW() - INTERVAL '24 hours' ORDER BY received_at DESC`.

4.5 Battery & Signal Trends (Optional)

What to show:

- Line chart of battery % over time.
- Line chart of signal strength over time.

Data query:

- `SELECT received_at, battery, signal_strength FROM tracker_packets WHERE imei = ? AND packet_type = 'status' AND received_at >= NOW() - INTERVAL '7 days' ORDER BY received_at ASC`.

5. Data Refresh Strategy

The tracker sends packets at configurable intervals (default 60 seconds for heartbeats, GPS on demand). The dashboard should:

- **Poll the database every 5–10 seconds** for live updates.
- Use **WebSockets or Server-Sent Events** if you want real-time push instead of polling.
- Cache aggressively for the map (e.g., refetch device list every 10s, but only refetch history when a device is selected).

---

6. GPS vs. WiFi/LBS — What the User Sees

Source	Accuracy	When It Happens
GPS (`packet_type = 'gps'`)	~5–10 meters	Device has satellite fix; best case.
WiFi/LBS (`packet_type = 'wifi_lbs'`)	~20–500 meters	Device cannot see GPS satellites; sends nearby WiFi routers and cell towers to Google for triangulation.
No location	—	Device has no GPS, no WiFi, and no cell towers; `latitude`/`longitude` will be NULL.

UI recommendation: Show a small indicator on each marker or in the detail panel explaining the fix type. For WiFi/LBS, you can optionally show the Google accuracy radius if you store it (currently not stored in the DB, but could be added to `tracker_packets` if needed).

---

7. Connection Details for the UI Team

Database Access

Your frontend should **not** connect directly to PostgreSQL from the browser. Instead, build a thin backend API (Node.js, Python, etc.) that:

1. Connects to the same `DATABASE_URL` as the receiver.
2. Exposes REST endpoints (or GraphQL) for the queries above.
3. Serves the static frontend build.

Environment Variables Needed

Variable	Source	Purpose
`DATABASE_URL`	Same as receiver	PostgreSQL connection string

No Google Maps API key is needed on the frontend — the receiver already handles geolocation and stores lat/lon in the database.

Generated by Kimi.ai

Suggested Tech Stack (Flexible)

- **Map:** Leaflet.js, Mapbox GL JS, or Google Maps JavaScript API.
- **Frontend:** React, Vue, Svelte, or plain HTML/JS.
- **Backend:** Express.js, Fastify, Flask, FastAPI, or Next.js API routes.
- **Real-time:** WebSockets, Server-Sent Events, or polling.

---

8. Edge Cases to Handle

Scenario	How to Handle
Device has never sent GPS	Marker not shown. Show "No location yet" in device list.
Device goes offline	`last_seen` stops updating. Fade the marker or change its color.
SOS with no location	Show the alert banner with IMEI and timestamp, but no map marker.
Duplicate packets	The receiver saves everything. Your UI should deduplicate by `id` or time window if needed.
Very old data	Default to 24-hour views. Allow the user to pick custom time ranges.
Many devices	Cluster markers on the map (e.g., Leaflet.markercluster) if > 50 devices.

---

9. Raw Protocol Notes (For Debugging Only)

If the UI team needs to show raw packet data for debugging:

- The `raw_hex` column contains the full binary packet as a hex string.
- Frame format: `0x78 0x78` (start) + length + protocol byte + content + `0x0D 0x0A` (stop).
- You do not need to parse this in the UI; it is for support/engineering diagnostics.

---

10. Summary Checklist for the UI Team

- Connect to the same Neon PostgreSQL database (via backend API).
- Build a device list from `tracker_devices`.
- Show map markers only from rows where `latitude` and `longitude` are not NULL.
- Color-code markers by `packet_type` (green = GPS, blue = WiFi/LBS).
- Draw polylines for selected device history (24h default).
- Show battery, signal, speed in a detail panel.
- Implement an SOS/drop alarm banner with sound/notification.
- Poll for updates every 5–10 seconds or use WebSockets.
- Handle offline devices gracefully (fade markers, show stale time).

Generated by Kimi.ai

11. Contact

For questions about packet types, database schema, or backend behavior, refer to the `GF01Handler` class in the receiver's `app.py`.